

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

- **Authors:** Google Research, Brain Team
- **arXiv:** [2010.11929](https://arxiv.org/abs/2010.11929)
- **한 줄 요약:** CNN의 image-specific inductive bias(locality, translation equivariance) 없이, 이미지를 patch sequence로 취급해 표준 Transformer encoder에 그대로 넣어도, 충분히 큰 규모(JFT-300M 등)로 pre-training하면 CNN(ResNet/Bit)을 능가하거나 대등한 성능을 낸다는 것을 실증한 논문

1. Introduction

Transformer는 NLP에서 model of choice가 되었고, large corpus pre-training + downstream fine-tuning 방식으로 100B parameter 이상 모델까지 스케일링되어 왔다.

Vision domain에서는 여전히 convolutional architecture(ResNet 계열)가 지배적이었고, self-attention은 CNN과 결합되거나 CNN의 일부 컴포넌트를 대체하는 정도로만 쓰이고 있었다. 이유는 naive self-attention을 픽셀 단위로 적용하면 pixel 수에 대해 quadratic cost가 발생해 현실적인 해상도에서 스케일이 불가능하기 때문.

기존 연구들은 이를 우회하기 위해 local self-attention, sparse attention, axial attention 등 specialized attention pattern을 도입했지만 hardware accelerator 상에서 효율적으로 구현하기 위해 복잡한 engineering이 필요하다는 한계가 있었다.

이 논문은 그래서

이미지를 patch 단위로 잘라서 그 patch들을 NLP의 word token처럼 취급하면, 별도의 specialized attention mechanism 없이도 표준 Transformer를 그대로(vanilla하게) 적용할 수 있다고 주장함

다만 이 방식은 CNN이 갖는 locality, 2D neighborhood structure, translation equivariance 같은 inductive bias를 거의 포기하는 것이기 때문에, 중간 규모 데이터셋(ImageNet 등)에서는 오히려 ResNet보다 성능이 낮다. 실험적으로 보여준 것은,

데이터 규모가 충분히 커지면(14M~300M images) 이 inductive bias 부족이 오히려 문제가 되지 않고, large scale training trumps inductive bias

2. Related Work

- Transformer는 원래 기계번역을 위해 제안되었고, BERT(denoising 기반 self-supervised pre-training), GPT 계열(language modeling 기반 pre-training)로 NLP의 표준이 되었다.
 - Naive self-attention을 이미지에 적용
 - pixel마다 다른 모든 pixel에 attend해야 해서 quadratic cost가 발생
 - 여러 approximation이 시도됨
 - local neighborhood self-attention
 - Sparse Transformer
 - block 단위 attention
 - axial attention

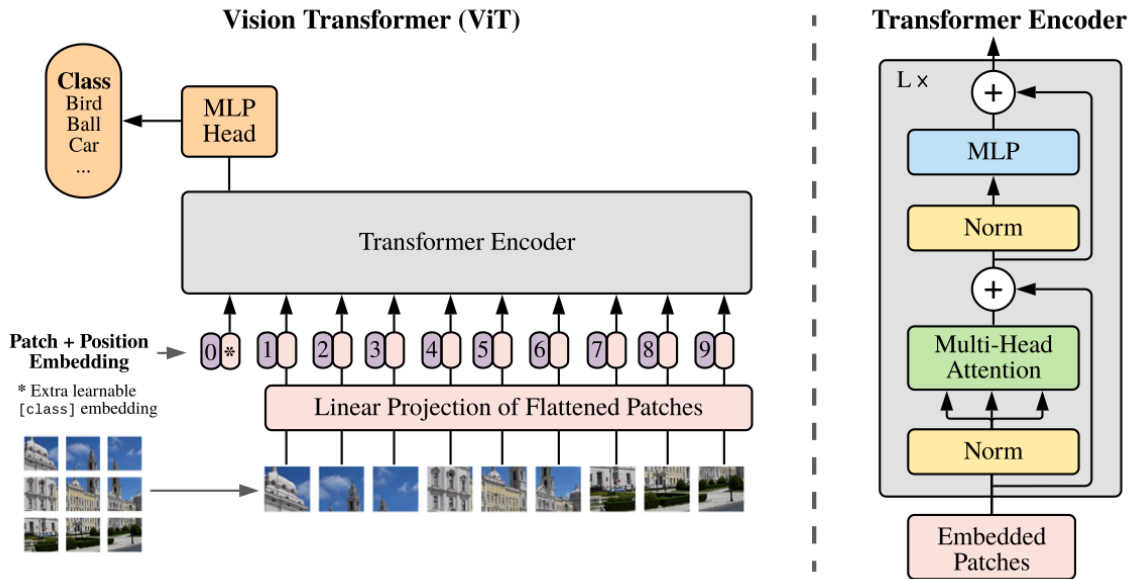
accelerator에 효율적으로 구현하려면 복잡한 engineering이 필요
 - 2×2 patch를 추출해 그 위에 full self-attention을 적용하는 모델 by Cordonnier et al. (2020)
 - ViT와 매우 유사하지만, 본 논문은
 - (i) large-scale pre-training이 vanilla Transformer를 SOTA CNN과 경쟁 가능하게 만든다는 것을 실증했고
 - (ii) Cordonnier et al.의 2×2 patch는 저해상도 이미지에만 적용 가능한 반면 ViT는 medium-resolution image까지 다룬다는 점에서 차별화된다.
 - CNN과 self-attention을 결합하는 다른 연구들(object detection의 DETR, video processing의 non-local network, unified vision-language model 등)
 - image GPT(iGPT, Chen et al., 2020a — pixel 단위로 Transformer를 generative하게 학습), ImageNet에서 최대 72% 정확도로, ViT의 supervised 방식보다는 낮다.
 - CNN을 대규모 데이터(ImageNet-21k, JFT-300M)로 transfer learning하는 선행 연구(BiT: Kolesnikov et al., 2020)의 데이터셋 세팅을 그대로 활용 → ResNet 대신 Transformer를 학습
-

3. Method

- NLP에서 이미 검증된 scalable architecture(Transformer)와 그 efficient implementation을 거의 out-of-the-box로 재사용할 수 있다는 것 자체가 이 논문의 설계 철학

- ViT의 novelty는 새로운 attention mechanism을 만드는 데 있지 않고, 이미지를 어떻게 Transformer가 먹을 수 있는 **sequence 형태로 바꿀 것인가**라는 **tokenization/embedding** 문제에 집중되어 있다.

3.1 Vision Transformer (ViT)



(1) Patch 분할과 flatten — 2D → 1D sequence 변환

표준 Transformer는 1D sequence of token embedding을 입력으로 받는다. 그런데 이미지는 본질적으로 2D(혹은 channel까지 포함하면 3D) 구조를 갖는다. 이 gap을 메우기 위해, 이미지

$\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ 를 flattened 2D patch들의 sequence $\mathbf{x}_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$ 로 reshape한다. 여기서:

- (H, W) : 원본 이미지 해상도
- C : channel 수 (RGB면 3)
- (P, P) : 각 patch의 해상도 (patch size)
- $N = HW/P^2$: patch 개수. 이 N 이 곧 Transformer의 **effective input sequence length**가 된다.

여기서 놓치기 쉬운 점은, $N = HW/P^2$ 이라는 관계식 때문에 **patch size가 작아질수록 sequence length가 quadratic하게 증가**한다는 점. Self-attention의 계산 비용이 $O(N^2)$ 이므로, patch size를 절반으로 줄이면($P \rightarrow P/2$) sequence length는 4배가 되고 attention 연산량은 16배가 된다. 이것이 ViT-L/16처럼 patch size를 모델 이름에 명시하는 이유이자, 16×16 words라는 논문 제목이 나온 배경

즉 하나의 16×16 patch가 NLP의 한 word token에 대응하는 단위라는 비유

(2) Patch Embedding — linear projection

Transformer는 모든 layer에서 constant latent vector size D 를 사용하므로 flatten된 patch ($P^2 \cdot C$ 차원 vector)를 D 차원으로 매핑해주는 **trainable linear projection $\mathbf{E}$** 가 필요하다:

$$\mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}$$

이 projection의 출력 $\rightarrow$ **patch embedding**

- 사실상 kernel size P , stride P 인 convolution 한 layer와 수학적으로 동일
 - non-overlapping하게 이미지를 훑으면서 각 patch를 D 차원으로 project하는 연산이기 때문

(3) [class] token — BERT에서 차용한 aggregation mechanism

BERT의 [CLS] token과 동일한 아이디어로, embedded patch sequence 앞에 **learnable embedding $\mathbf{x}_{\text{class}}$** 를 prepend한다:

$$\mathbf{z}_0^0 = \mathbf{x}_{\text{class}}$$

- 이 토큰은 순수하게 학습되는 파라미터
- Transformer encoder를 통과한 후의 최종 상태 $\mathbf{z}_L^0$ 이 곧 이미지 전체의 representation $\mathbf{y}$ 로 쓰인다.
- Pre-training과 fine-tuning 모두에서 classification head가 이 $\mathbf{z}_L^0$ 에 부착되는데, pre-training 시에는 hidden layer 하나를 가진 MLP, fine-tuning 시에는 single linear layer로 구현

왜 굳이 별도의 class token을 두고 global average pooling을 쓰지 않았는지? 순수하게 원본 Transformer 디자인을 그대로 따르고자 했기 때문, appendix에서 GAP과 class token의 성능 차이가 크지 않음을 별도로 논의(learning rate 설정이 더 중요한 factor였다고 보고).

(4) Position Embedding — 2D 구조 정보의 주입

Patch embedding에는 순서/위치 정보가 전혀 없다(self-attention 자체는 permutation-invariant하기 때문). 따라서 positional information을 보존하기 위해 position embedding $\mathbf{E}_{\text{pos}}$ 을 더해준다:

$$\mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$$

- 여기서 $N + 1$ 인 이유는 class token 자리까지 포함하기 때문.
- 2D-aware positional embedding(예: patch의 row, column을 별도로 encoding)을 쓰지 않고 **standard learnable 1D position embedding**을 사용함!
 - appendix D.4에서 2D-aware variant를 시도했지만 유의미한 성능 향상을 관찰하지 못했다고 보고
 - 이는 후에 4.5절(Inspecting ViT)에서 확인되는데, 1D position embedding만으로도 모델이 학습 과정에서 스스로 row-column 구조와 distance 관계를 내재적으로 학습해낸다(즉 2D topology를 데이터로부터 emergent하게 복원한다).

(5) 전체 embedding 수식 — Eq. (1)

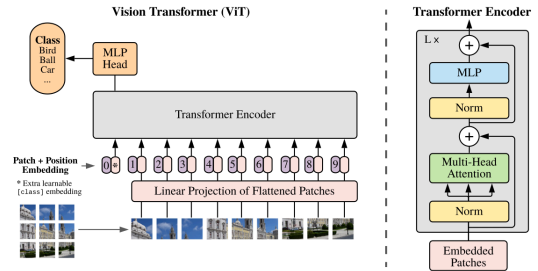
위 세 요소(class token, patch embedding, position embedding)를 합치면

Transformer encoder에 들어가는 최초 입력 $\mathbf{z}_0$:

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{pos}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \quad \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$$

(6) Transformer Encoder — Pre-LN 구조

- Encoder는 Multiheaded Self-Attention과 MLP block이 번갈아 쌓인 구조.
- **Layernorm**이 각 block 이전에 적용되고, **residual connection**은 각 block 이후에 더해진다(Pre-LN 방식). 학습 안정성 때문



$$\mathbf{z}'_{\ell} = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \quad \ell = 1 \dots L \quad (2)$$

$$\mathbf{z}_{\ell} = \text{MLP}(\text{LN}(\mathbf{z}'_{\ell})) + \mathbf{z}'_{\ell}, \quad \ell = 1 \dots L \quad (3)$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0) \quad (4)$$

MLP는 두 개의 layer와 GELU 비선형 함수로 구성. 최종 image representation $\mathbf{y}$ 는 마지막 layer의 class token $\mathbf{z}_L^0$ 에 LN을 적용한 것.

Appendix A: Multihead Self-Attention 수식도 함께 짚으면 입력 sequence $\mathbf{z} \in \mathbb{R}^{N \times D}$ 에 대해:

$$[\mathbf{q}, \mathbf{k}, \mathbf{v}] = \mathbf{z} \mathbf{U}_{qkv}, \quad \mathbf{U}_{qkv} \in \mathbb{R}^{D \times 3D_h} \quad (5)$$

$$\mathbf{A} = \text{softmax} \left(\mathbf{q} \mathbf{k}^\top / \sqrt{D_h} \right), \quad \mathbf{A} \in \mathbb{R}^{N \times N} \quad (6)$$

$$\text{SA}(\mathbf{z}) = \mathbf{A} \mathbf{v} \quad (7)$$

k 개의 head를 병렬로 돌리고 concat 후 다시 project하는 것이 MSA:

$$\text{MSA}(\mathbf{z}) = [\text{SA}_1(\mathbf{z}); \text{SA}_2(\mathbf{z}); \dots; \text{SA}_k(\mathbf{z})] \mathbf{U}_{msa}, \quad \mathbf{U}_{msa} \in \mathbb{R}^{k \cdot D_h \times D} \quad (8)$$

Head 수 k 를 바꿔도 전체 compute/parameter 수를 일정하게 유지하기 위해 관례적으로 $D_h = D/k$ 로 설정

(7) Inductive Bias — ViT가 CNN과 근본적으로 다른 지점

- CNN은 **locality, 2D neighborhood structure, translation equivariance**를 매 layer마다 architecture 자체에 built-in
 - convolution kernel이 local receptive field를 가지고, 동일한 kernel이 모든 위치에서 공유(weight sharing)되기 때문에 translation equivariant.
- 반면 ViT에서는:
 - MLP layer만 local하고 translationally equivariant하다(position-wise로 독립 적용되므로)
 - Self-attention layer는 global하다 (첫 layer부터 모든 patch가 서로 attend할 수 있다)
 - 2D neighborhood structure는 딱 두 곳에서만 manually 주입된다
 - 맨 처음 이미지를 patch로 자르는 단계
 - fine-tuning 시 해상도가 바뀔 때 position embedding을 2D interpolation하는 단계.
 - 그 외에는 position embedding이 초기화 시점에는 2D 위치에 대한 아무 정보도 담고 있지 않거 patch들 간의 모든 spatial relation은 학습을 통해 from scratch로 습득되어야 함

Inductive bias는 일종의 **prior** 역할을 하는데, **prior**가 데이터와 잘 맞으면 **sample efficiency**를 높여주지만, 데이터가 충분해 모델이 스스로 규칙성을 배울 수 있다면 오히려 그

prior가 표현력을 제약하는 constraint로 작용할 수 있다는 **bias-variance/inductive bias trade-off**의 전형적인 사례

(8) Hybrid Architecture — CNN feature map을 patch 대신 사용

- Raw image patch 대신 CNN의 feature map에서 patch를 추출해 input sequence를 구성할 수도
- 이 경우 patch embedding projection $\mathbf{E}$ 는 raw pixel patch가 아니라 CNN feature map에서 추출된 patch에 적용
 - 극단적인 특수 케이스로 patch spatial size를 1×1 로 설정하면 이는 CNN feature map의 spatial dimension을 단순히 flatten해서 Transformer 차원으로 project하는 것과 동일
 - 이 hybrid 방식에서도 class token과 position embedding은 앞서 설명한 방식 그대로 추가된다.
- 실험적으로(4.3, 4.4절) hybrid 모델은 작은 computational budget에서는 순수 ViT보다 약간 더 나은 성능을 보이지만, 모델 크기가 커질수록 그 차이는 사라진다. 이는 convolutional local feature processing이 작은 스케일에서는 여전히 도움이 되지만, 충분히 큰 스케일에서는 Transformer가 스스로 그런 local structure를 데이터로부터 학습해낼 수 있음을 시사

3.2 Fine-tuning and Higher Resolution

일반적인 워크플로우는 ViT를 대규모 데이터셋으로 pre-train한 뒤, 더 작은 downstream task로 fine-tuning하는 것. Fine-tuning 시에는:

1. Pre-trained prediction head를 제거하고, zero-initialize된 $D \times K$ feedforward layer를 새로 부착한다 (K = downstream class 수).
2. Pre-training보다 더 높은 해상도로 fine-tuning하는 것이 유익하다는 선행 연구를 따른다.
3. 해상도를 높일 때 patch size는 그대로 유지하므로, 결과적으로 effective sequence length N 이 커진다 ($N = HW/P^2$ 이므로 H, W 가 커지면 N 도 커짐). ViT는 임의의 sequence length를 다룰 수 있지만, 문제는 pre-trained position embedding이 원래 sequence length에 맞춰 학습되었기 때문에 더 이상 의미가 맞지 않게 된다는 점.
4. 이를 해결하기 위해 pre-trained position embedding에 대해 원본 이미지 상에서의 위치를 기준으로 2D interpolation을 수행한다.

저자들은 이 해상도 조정과 최초의 patch extraction, 이 두 지점만이 ViT 안에서 2D 이미지 구조에 대한 inductive bias가 manually 주입되는 유일한 지점이라고 명시적으로 강조(최소한의 vision-specific 수정만 가했음을 명시)

4. Experiments

- **Model variants:** BERT 설정을 따라 ViT-Base(12 layer, D=768, 86M params), ViT-Large(24 layer, D=1024, 307M params), ViT-Huge(32 layer, D=1280, 632M params) 세 가지를 정의. 표기는 ViT-L/16 처럼 모델 크기/patch size 로 나타낸다.
- **Datasets:** ImageNet(1.3M), ImageNet-21k(14M), JFT-300M(303M, in-house) 세 스케일로 pre-training하고 → ImageNet/ReaL, CIFAR-10/100, Oxford Pets/Flowers, VTAB(19-task) 등으로 transfer 성능을 평가.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-121k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

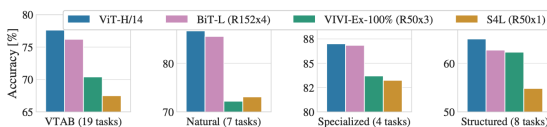
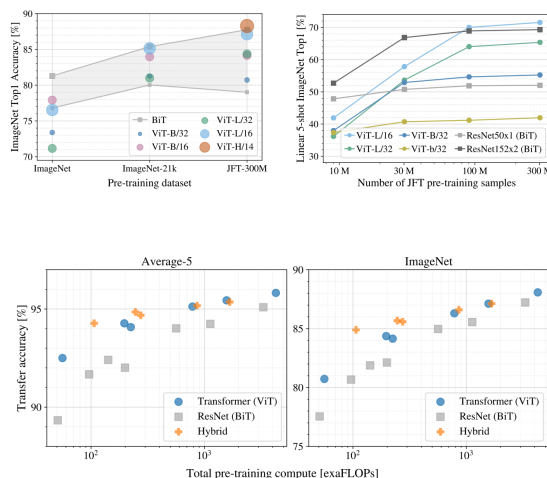


Figure 2: Breakdown of VTAB performance in *Natural*, *Specialized*, and *Structured* task groups.

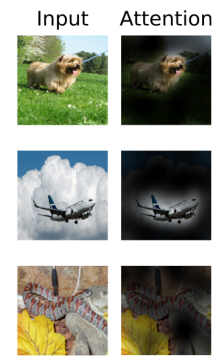


- **SOTA 비교(4.2):** JFT-300M으로 pre-train한 ViT-H/14, ViT-L/16이 BiT-L(ResNet152x4)과 Noisy Student(EfficientNet-L2)를 능가하거나 대등하면서, pre-training에 필요한 TPUv3-core-days는 훨씬 적었다 (ViT-L/16: 0.68k vs BiT-L: 9.9k vs Noisy Student: 12.3k).

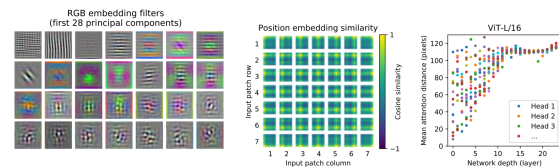
- **Pre-training data 요구량(4.3):** 작은 데이터셋(ImageNet)에서는 ViT-Large가 ViT-Base보다도 성능이 낮았지만, JFT-300M 규모에서는 큰 모델의 이점이 명확히 드러났다. 즉 **convolutional inductive bias**는 작은 데이터셋에서 유용하지만, 큰 데이터셋에서는 데이터로부터 직접 패턴을 학습하는 것으로 충분하고 오히려 더 유리하다는 것이 재확인된다.

- **Scaling study(4.4):** performance-vs-compute trade-off에서 ViT가 ResNet을 지배(dominate)하며, 동일 성능 대비 약 2-4배 적은 compute를 사용한다. Hybrid는 작은 budget에서만 약간 유리하다.

- **Inspection(4.5):** 학습된 patch embedding filter의 PCA를 보면 저차원 basis function처럼 보이고, position embedding은 학습을 통해 row-column 구조와 거리 개념을 스스로 encode하게 된다(2D-aware embedding을 수동 설계할 필요가 없었던 이유). Attention distance 분석 결과, 낮은 layer에서부터 이미지 전역에 attend하는 head가 존재함을 확인 — CNN의 receptive field 개념과 대비되는 “attention distance”가 실제로 CNN보다 훨씬 이른 layer에서 전역적으로 확장될 수 있음을 보여준다.



- **Self-supervision(4.6):** BERT의 masked language modeling을 모사한 masked patch prediction으로 self-supervised pre-training을 시도, ViT-B/16 기준 79.9%(scratch 대비 +2%, supervised 대비 -4%)를 달성. Contrastive pre-training은 future work로 남김.



5. Conclusion

- Transformer를 이미지 인식에 직접 적용하면서, 최초의 patch extraction 외에는 image-specific inductive bias를 architecture에 추가하지 않았다는 것이 이 논문의 핵심 기여.
- 단순하지만 scalable한 전략이 대규모 데이터셋 pre-training과 결합되면 잘 작동.
- 향후 과제
 - detection, segmentation 등 다른 vision task로의 확장(DETR 계열, Swin Transformer, ViTDet 등으로 실제 이어짐)
 - self-supervised pre-training 방법 탐색
 - 추가적인 scaling

6. ViT의 Patchify 개념을 다른 도메인(특히 AI4Science)으로 확장하기

6.1 Patchify의 본질을 다시 보기

patchify:

연속적이고 고차원적인 raw signal을 local chunk로 자르고, 그 각각을 linear (또는 shallow) projection으로 embedding하여 discrete token sequence로 변환하는 절차 modality-agnostic한 tokenization primitive로 일반화될 수 있으며, 실제로 time series(PatchTST), video, point cloud, molecular/graph 데이터 등 다양한 도메인으로 확장되어 왔다.

AI4Science 맥락에 맞춰 생각해보면, patchify가 던지는 질문은 결국 다음과 같이 재구성된다:

규칙적인 grid(pixel)가 아니라 irregular, unordered, sparse한 구조(원자, 포인트, 그래프 노드)를 어떻게 고정 크기의 local group으로 자르고, 그 그룹을 하나의 token으로 embedding할 것인가?

이미지에서는 patch가 정의상 자명하지만(grid 상의 $P \times P$ 사각형), 분자나 point cloud에서는 이 patch에 해당하는 grouping 방식 자체가 non-trivial한 설계 문제가 된다.

6.2 실제 응용/발전 사례

1. Point cloud — Point-BERT / Point-MAE (patchify를 FPS + k-NN grouping으로 재정의)

- Point cloud는 순서가 없고 밀도가 불균일한 3D point set
- Point-BERT 계열은 ViT의 정사각형 grid patch 대신, **farthest point sampling(FPS)**으로 center point들을 뽑고, 각 center의 k-NN를 grouping하여 하나의 point patch(sub-cloud)로 정의한다. 이렇게 만든 point patch를 mini-PointNet류의 tokenizer로 embedding하면, 이후는 표준 Transformer encoder에 그대로 넣을 수 있다
- 이 모델은 입력 point cloud로부터 patch embedding sequence를 추출해 표준 Transformer encoder에 전달한다.

2. Point cloud — Pix4Point / PViT (image-pretrained ViT의 backbone을 그대로 transfer)

- ImageNet으로 pre-train된 ViT backbone 자체(weight까지)를 3D point cloud task로 transfer하는 시도, 네트워크 backbone 구조를 변경하지 않고도 3D 이해 성능을 끌어올린다.
- Tokenizer만 도메인에 맞게(progressive point patch embedding, graph convolution 기반) 새로 설계하고 Transformer 본체는 그대로 재사용
- **tokenizer(입력을 어떻게 자르고 embedding할지)와 backbone(무엇으로 처리할지)을 분리해서 설계하는 사고방식**

3. 분자 그래프 — Image-Like Graph Representation (CubeMol) 방식

- 분자를 grid/cube 형태의 image-like representation으로 투영한 뒤, ViT를 거의 그대로 적용하는 접근
- CubeMol 표현으로부터 정보를 모으는 데 self-attention이 CNN보다 적합하다고 보고 ViT 구조를 채택
- non-overlapping patch 외에 patch 경계(edge)와 교차점(inner) 영역을 별도로 다루는 추가 patch 타입을 도입 → 분자 그래프의 edge connectivity를 patch 설계에 반영한 사례